From Structural Mapping to Intention Reasoning: A Review of LLM-based Mobile Test Migration

TIAN GAN, Technical University of Munich, Germany

Testing mobile applications across heterogeneous Android devices remains challenging due to ecosystem fragmentation and the brittleness of UI-dependent automation techniques. Traditional test migration approaches rely heavily on static UI attributes, such as XML resource identifiers, which limits their robustness across structurally dissimilar applications.

This seminar report examines how recent Large Language Model (LLM)–based approaches address this limitation by shifting test migration from structure-level mapping to intention-level reasoning. By synthesizing recent literature, I categorize the migration process into a unified three-stage pipeline: abstraction of test intent, planning of execution strategies, and grounding of abstract actions onto concrete UI elements.

While LLM-based frameworks demonstrate improved flexibility and cross-app adaptability, the analysis reveals persistent challenges, including hallucination, visual grounding errors, and execution instability. Finally, this report discusses emerging industrial practices, such as applications at Amazon, and argues that hybrid approaches combining LLM reasoning with traditional verification mechanisms are likely necessary for reliable large-scale deployment.

Additional Key Words and Phrases: Mobile App Testing, Large Language Model, Automated Test Migration

1 Introduction

Mobile applications are routinely deployed across a wide range of Android devices with varying screen sizes, system versions, and hardware configurations. While automated GUI testing is widely used to ensure functional correctness and support regression testing, maintaining such tests in practice remains costly. Even minor UI changes—such as layout adjustments, renamed resource identifiers, or redesigned navigation flows—can cause existing test scripts to break, requiring substantial manual effort to repair or rewrite them [5, 7].

To reduce this maintenance burden, prior research has explored automated test migration, which aims to reuse or adapt test cases from one application to another with similar functionality [7, 18]. Traditional approaches typically rely on static UI properties, such as XML resource identifiers, widget hierarchies, or predefined mappings between screens [5, 7]. Although effective in controlled settings, these techniques often struggle when applied to real-world applications that exhibit diverse workflows, dynamic content, or non-standard UI designs [7, 18]. As a result, test migration remains brittle when structural similarity between applications is limited.

Recently, Large Language Models (LLMs) have emerged as a promising alternative for addressing these limitations [2, 3, 5, 16]. Unlike earlier methods that focus primarily on structural correspondence, LLM-based approaches attempt to reason about the intention underlying a test case—for example, verifying a successful login or adding an item to a shopping cart—and then adapt this intention to a new application context [2, 3, 17]. By leveraging their ability to model semantic relationships and procedural logic, LLMs enable a fundamental shift from structure-level mapping toward intention-level reasoning [2, 5, 16].

However, adopting LLMs does not eliminate all challenges. While recent frameworks demonstrate improved flexibility, they introduce new sources of uncertainty [6, 10]. Issues such as hallucinated actions, incorrect visual grounding, and unstable execution behavior have been repeatedly reported in the literature [1, 6, 10, 14]. These limitations raise important questions about the reliability, scalability, and practical deployment of LLM-based systems.

Author's Contact Information: Tian Gan, tian.gan@tum.de, Technical University of Munich, Germany.

The **remainder** of this report is structured as follows. Section 2 provides background on automated mobile application testing and reviews traditional test migration approaches prior to the adoption of Large Language Models. Section 3 analyzes recent LLM-based test migration frameworks and synthesizes their methodologies into a unified abstraction–planning–grounding pipeline. Section 4 discusses the key challenges and open problems associated with LLM-driven migration, including semantic drift, visual grounding errors, and evaluation limitations. Section 5 outlines future research directions and examines emerging industrial practices, with a particular focus on large-scale deployment scenarios. Finally, Section 6 concludes the report and summarizes the main findings.

2 Background

2.1 Pre-LLMs Approaches

Prior to the emergence of Large Language Models, automated test migration primarily relied on heuristic rules and static analysis. These approaches can be broadly categorized into **Mapping-based** [6] and **Search-and-Synthesis-based** methods [18].

- **Mapping-based Approach:** Represented by frameworks like **CraftDroid**, this category focuses on transferring test cases and oracle through semantic mapping. **CraftDroid** operates via a three-component pipeline [7]:
  - **Test Augmentation** instruments the source app, augments the original test cases to extract textual information from the exercised GUI widgets.
  - **Model Extraction** employs static analysis on the target app to map transitions between Activity components, constructing a UI Transition Graph (UITG).
  - **Test Generation** calculates the similarity between source and target widgets. It uses textual information and location of widgets in the UITG. The calculation relies on NLP token matching and a (Word2Vec)-based weighting algorithm. Finally it generates new tests, transfers oracles (including negative checks) and verifies path reachability on the target app.

  While this architecture proved effective for applications with simple textual correspondence , it suffers from a lack of contextual understanding. Because the method processes widgets in isolation, it fails to capture the sequential logic inherent in complex test scenarios. Consequently, the mapping mechanism often breaks when target applications employ textless icons or divergent workflows. Furthermore, the over-reliance on static UITG performs poorly on complex and large-size apps. For instance, transfer success rates drop significantly in complex domains such as shopping applications [7].
- **Search-and-Synthesis-based Approach:** Conversely, frameworks like **Appflow** utilize supervised machine learning to explore the application's state space rather than relying on direct mapping. By recognizing screens and widgets, it synthesizes tests through traversing a pre-defined behavior graph. This methodology is primarily structured into two phases [5]:
  (1) **Offline Learning:** It utilizes large-scale labeled datasets to train deep learning classifiers. By analyzing large-scale dataset of GUI screenshots and layout information, it identifies common abstract screens and abstract widgets. This effectively decouples test execution from underlying resource IDs.
  (2) **Online Synthesis:** Via traversing a pre-defined Behavior Graph that describes the states of screens and actions, it dynamically maps abstract user paths to concrete widgets on the current screen, guiding the exploration of the application's state space.

Although AppFlow demonstrated strong resilience to UI layout changes and fragile resource IDs, it remains limited by its inability to understand test intent. Critical drawbacks include [7]

(1) a Lack of context-awareness in inputs;
(2) Limited expressiveness in test generation.
(3) Absence of oracles.

Moreover, the heavy reliance on supervised learning restricts domain generalization. The approach is often confined to specific app categories and cannot adapt to new genres without retraining on extensive datasets.

2.2 Overview of LLM

LLMs have recently become an important foundation for software engineering tasks involving code understanding and transformation. Trained on large-scale corpora of natural language and source code, LLMs are able to capture both syntactic structures and higher-level semantic relationships between code elements[16]. This property is particularly relevant to automated test script migration, where preserving test intent and execution logic across heterogeneous mobile applications is a central challenge.

From a task-oriented perspective, automated test script migration can be viewed as a sequence-to-sequence transformation problem: a source test script encodes a sequence of user actions and assertions, while the target script aims to reproduce equivalent functionality under different UI structures, identifiers, or workflows. Previous migration approaches rely heavily on manually engineered abstractions or similarity heuristics, which limits their robustness and generalization ability [5, 7]. In contrast, LLMs provide a data-driven alternative by learning implicit correspondences between code, textual context, and execution semantics directly from training data.

According to recent surveys on LLMs for software engineering, two capabilities are particularly relevant to test script migration: code generation and code translation [16]. Code generation enables LLMs to synthesize executable test scripts in a target framework while adapting API calls, UI locators, and control structures. Code translation allows LLMs to transform existing scripts across applications or testing environments while preserving functional intent, even when UI hierarchies and resource identifiers differ substantially. Compared to traditional AST-based techniques, these capabilities operate at a semantic level and are better suited to handling dynamic identifiers and structurally dissimilar interfaces [16].

Overall, the integration of LLMs shifts test script migration from manually designed abstraction rules toward intent-aware adaptation. While this introduces new challenges such as controllability and reproducibility, recent work suggests that LLM-based approaches can reduce manual effort and improve the maintainability of migrated test scripts when combined with appropriate validation mechanisms [16].

3 Methodology

3.1 Methodological Framework Overview

From a methodological perspective, the integration of Large Language Models (LLMs) into test migration represents more than just a technical upgrade; it marks a fundamental paradigm shift from structure-level mapping to intention-level reasoning. Unlike traditional approaches that relied heavily on rigid heuristic rules [7], modern frameworks operate by decoupling the abstract test intention from its concrete implementation. While architectural choices vary across recent studies—such as **LLMigrate** [2], **ReuseDroid** [6], and **MigratePro**—the general methodology converges into a three-stage pipeline: Abstraction, Planning, and Grounding.As illustrated in Fig.

1, this pipeline provides a conceptual overview of how source test traces are transformed into executable actions on heterogeneous target UIs through intention reasoning and visual grounding.

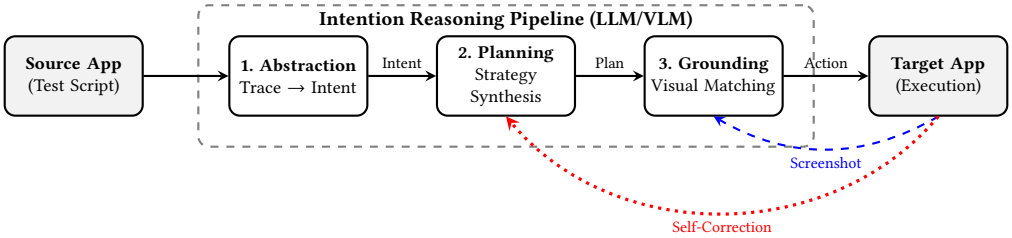

Fig. 1. Conceptual abstraction of LLM-driven test migration pipelines. The workflow summarizes recurring components reported in prior work, emphasizing intent reasoning, planning, and visual grounding, together with feedback loops commonly adopted for adaptive execution.

- Step 1: **Comprehension and Abstraction:**
  A major hurdle in test migration lies in the significant structural divergence between source and target applications, such as differing XML hierarchies and resource IDs. To address this, current frameworks typically leverage LLMs to transform low-level execution traces into a platform-agnostic Intermediate Representation (IR). For instance, **ITEM** [3] summarizes these traces into natural language intentions (e.g., "Search for product 'X'"), while **MACdroid** [17] attempts to mitigate bias by extracting a "General Test Logic" derived from multiple apps.
  However, using natural language as an IR is not without risks. It introduces a layer of ambiguity, as the same intention description might be interpreted differently by downstream planners, especially when domain-specific constraints are implicit rather than explicit. Consequently, the quality of abstraction in this initial stage effectively sets the upper bound for the success of the entire migration process.
- Step 2: **Planning and Reasoning:**
  Once an abstract intention is formulated, the system faces the challenge of bridging the semantic gap to the unseen target UI. This stage necessitates complex reasoning, particularly to resolve the **1-to-N mapping problem**, where a single abstract step like "Login" might require a multi-step interaction sequence in the target app due to workflow discrepancies. Approaches in the literature differ significantly here. **ReuseDroid** [6] utilizes a Planner Agent to decompose high-level goals into atomic actions based on the current UI state. In contrast, **MigratePro** adopts a synthesis-based approach, building a State Graph to explore and generate entirely new paths. From a critical standpoint, this highlights a fundamental trade-off: while planner-based approaches tend to preserve the original test semantics more faithfully, synthesis-based approaches appear to offer higher adaptability across structurally dissimilar apps, albeit at the cost of potential deviation from the original test scenario.
- Step 3: **Visual Grounding and Execution:**
  The final stage involves Visual Grounding, where abstract plans are anchored to concrete UI elements. To mitigate the "Hallucination vs. Reality" gap—especially when accessibility metadata is missing—frameworks like **ScreenAI** [1] integrate Vision-Language Models (VLMs) to identify elements based on visual features.
  In practice, this stage arguably represents the most fragile link in the pipeline. Unlike planning errors, which can often be recovered through re-reasoning, grounding errors (such

as false positive element detection) are harder to detect and can silently invalidate the test execution. Furthermore, the non-deterministic nature of VLM outputs introduces a degree of instability that remains a significant challenge for industrial adoption.

3.2 Enabling Technologies and Trade-offs

To support this open-ended reasoning process, several technical components are essential, though they each introduce specific limitations.

Prompt Engineering and Context Limits. Techniques such as **In-Context Learning** and **Chain-of-Thought (CoT)** [11] are widely employed to bridge the cognitive gap between human intent and machine execution. However, these strategies are inevitably bounded by the Context Window Limit. Including extensive UI hierarchy data and few-shot examples often exhausts the token budget, which may force a trade-off between the depth of context provided and the complexity of the reasoning required.

Multimodal Context Management. While feeding screenshots directly to Multimodal LLMs compensates for the lack of reliable text attributes, it incurs significant computational costs. Moreover, the necessity of resizing high-resolution screenshots to fit model constraints can lead to the loss of fine-grained visual details, making it difficult for the model to interact with smaller UI elements.

Self-Correction Loops. Frameworks like **LLMigrate** [2] attempt to mitigate these issues by implementing feedback loops where execution errors trigger re-planning. While this significantly improves robustness compared to static scripts, it is not a panacea; from the reported results, it appears that systems can still fall into infinite correction loops if the underlying reasoning logic is fundamentally flawed.

Summary: Together, these components enable LLM-based migration frameworks to operate in dynamic, open-ended GUI environments. However, they also shift the primary challenge from manual script maintenance to managing the efficiency, robustness, and stochastic nature of AI-driven agents.

4 Challenges and Open Problems

Despite the promising methodology outlined in Section 3, recent LLM-based test migration frameworks continue to face fundamental challenges that limit their robustness, scalability, and practical applicability. Importantly, these challenges are not isolated to individual components but emerge across the abstraction–planning–grounding pipeline, as consistently reported in both method-driven studies and recent survey work.

4.1 Ambiguity and Semantic Drift in Intention Abstraction

A core challenge arises during the abstraction stage, where low-level test scripts are transformed into intention-level representations. Approaches such as **MACdroid** [17] and **ITEM** [3] demonstrate that natural language IRs substantially improve cross-app generalization. However, they also risk obscuring implicit constraints embedded in the original tests, including UI state dependencies, ordering assumptions, and domain-specific rules.

This ambiguity becomes particularly problematic when semantically similar intentions correspond to divergent operational behaviors across apps (e.g., "Checkout" implying a full payment workflow in one app but merely a confirmation or summary view in another). As reported in semantic mapping–based transfer approaches, downstream planners may therefore generate executions that are syntactically valid yet semantically misaligned with the original test objective, leaving correctness largely implicit. In other words, the agent may successfully "click through" a flow while completely missing the business logic verification that the original test was designed to enforce.

4.2 Planning under Structural Uncertainty: Migration vs. Generation

Once an abstract intention is formed, the planning stage must bridge the semantic gap to an unseen target UI under conditions of structural and behavioral uncertainty. Agent-based approaches, such as those adopted in **LLMigrate** [2] and agent-centric systems surveyed in [6], rely on incremental reasoning over the perceived UI state. In practice, however, real-world apps frequently exhibit asynchronous content loading, hidden navigation paths, and context-dependent flows, which challenge the reliability of such planners.

To address these limitations, synthesis-based approaches—most notably those inspired by **AppFlow** [5] and **MigratePro**—explore the target app to construct new execution paths rather than strictly reusing source test steps. While this strategy improves adaptability to structurally dissimilar apps, it also exposes a fundamental methodological trade-off: as synthesized paths diverge from the original execution trace, the boundary between test migration and test generation becomes increasingly blurred.

This raises a critical open question that remains largely unaddressed in current literature: to what extent can a migrated test deviate from its source before it no longer constitutes a faithful transfer? For instance, if a synthesis-based tool bypasses a buggy login screen to reach the dashboard via a deep link, has it migrated the test—or merely evaded the very bug the test was intended to detect? While planner-based approaches emphasize semantic fidelity, synthesis-driven methods favor adaptability and coverage, often at the cost of preserving the original testing intent. Resolving this tension is central to defining the scope and validity of LLM-based test migration as a distinct research problem.

4.3 The Fragility of Visual Grounding in Non-Standard UIs

Visual grounding represents one of the most fragile stages in the migration pipeline, particularly in non-standard UI settings where accessibility metadata is incomplete, inconsistent, or entirely absent. Systems such as **ScreenAI** [1] and **ReuseDroid** [6] leverage VLMs to identify UI elements based on visual features, an approach that is unnecessary in well-structured, standard UIs but essential in modern, icon-heavy interfaces. However, as also observed in multimodal agent systems like **AppAgent** [15], such models remain highly sensitive to visual noise, icon similarity, and layout variations.

A particularly challenging failure mode is false positive element matching. Unlike planning errors, which often manifest as execution failures and trigger re-reasoning, grounding errors may silently redirect the test flow without explicit signals. Current frameworks lack robust confidence estimation or verification mechanisms to assess whether a grounded action truly corresponds to the intended UI element, significantly undermining execution reliability.

4.4 Feedback Loops, Efficiency, and Stability

Several frameworks, including **ReuseDroid** and **LELANTE** [4], incorporate feedback-driven self-correction loops to improve robustness in complex migration scenarios. Empirical results reported in these systems suggest that active feedback significantly outperforms static scripts in terms of success rate. However, this improvement comes at a notable cost to execution efficiency and stability.

Repeated re-planning and exploration can lead to excessive runtime or even infinite correction loops when abstraction or perception errors persist. More broadly, survey studies on LLM-based software testing indicate that agent-based exploration can be orders of magnitude slower than traditional script execution, raising concerns about scalability in industrial testing pipelines. These

observations suggest that current feedback mechanisms often react to execution symptoms rather than diagnosing underlying causes, limiting their effectiveness in practice.

4.5 The Crisis of Evaluation and Reproducibility

Beyond individual pipeline stages, evaluation remains a critical and largely unresolved issue. Recent surveys consistently point out that most existing studies evaluate migration performance on limited app sets, making it difficult to assess generalization across domains [10, 16]. Moreover, the stochastic nature of LLMs complicates reproducibility, as identical inputs may yield divergent execution traces.

Importantly, the lack of standardized benchmarks in this area should not be interpreted as an absence of evaluation efforts. On the contrary, several high-profile benchmarking environments have recently emerged. A prime example is **AndroidWorld** [9], which defines a dynamic environment for autonomous agents with real-world tasks. While such benchmarks represent significant progress, their evaluation criteria are largely outcome-oriented—focusing primarily on whether a final state is reached.

However, such evaluation paradigms are fundamentally misaligned with the goals of test migration. In this context, success is not merely reaching a terminal UI state, but preserving step-level test logic and verification intent. A migrated test that bypasses intermediate checks may achieve task completion while silently violating the original test objective. Consequently, benchmarks centered on task success rates, like those in [9], are insufficient to capture the semantic fidelity required for regression testing [14].

A related challenge arises in visual grounding. While models like **ScreenAI** [1] benefit from large-scale training on datasets like Android in the Wild, grounding errors remain prevalent in non-standard UIs. Recent work such as **Ferret-UI** [12] has begun to address this by introducing fine-grained metrics for UI comprehension. Nevertheless, as discussed in vision-based testing reviews [13], current migration frameworks have yet to systematically integrate such multimodal metrics, often failing to penalize "lucky" clicks where the flow continues despite incorrect element interaction.

More broadly, future evaluation frameworks must move beyond binary pass/fail outcomes. Insights from UI representation learning [13] suggest that embedding-based similarity measures could capture the semantic correspondence between source and target UI components. Addressing this gap will likely necessitate metrics that jointly consider task execution and step-level behavioral fidelity, rather than relying solely on end-task reachability.

4.6 Summary and Outlook

Taken together, these challenges indicate that LLM-based test migration is gradually shifting the field from deterministic script reuse toward adaptive, agent-driven testing. While this transition enables greater flexibility, it also introduces new concerns regarding semantic fidelity, evaluation validity, efficiency, and control. Addressing these open problems will likely require hybrid solutions that combine learned reasoning with symbolic constraints and explicit verification, rather than relying solely on end-to-end LLM behavior.

5 Future Directions and Industrial Outlook

While Section 4 highlights the current limitations of LLM-based test migration, emerging research and industrial developments suggest a trajectory toward improvement. Specifically, the critical challenges identified in Section 4—ranging from high operational costs and visual grounding fragility to the misalignment of evaluation benchmarks and the difficulty of long-horizon planning—motivate

a shift toward more autonomous, reliable, and standardized systems. Drawing from recent comprehensive surveys on vision-based testing and GUI agents [13, 14], alongside industrial case studies from Amazon[8], this section outlines the critical path forward.

5.1 Convergence of Vision and Language: The Multimodal Paradigm

The traditional separation between "vision-based testing" and "text-based scripting" appears to be increasingly blurred. As indicated by recent literature [13], a promising paradigm lies in Multimodal Perception, where agents bridge the semantic gap by reasoning about UI states purely from rendered pixels.

- **Addressing Structural Brittleness:** This multimodal approach directly mitigates the brittleness of structure-level representations discussed in Section 4, where XML- or view-hierarchy–based mappings often fail under minor UI changes.
- **Beyond Crash Detection:** Rather than focusing solely on crash detection, future agents are likely to evolve toward identifying subtle Non-Crash Bugs (e.g., logic errors, layout glitches) by combining visual pattern recognition with semantic understanding.
- **Cross-Platform Generalization:** By relying on visual cues rather than fragile OS-specific metadata, multimodal agents offer the potential for true Cross-Platform Invariance, allowing tests to adapt across mobile, web, and IoT interfaces with minimal human intervention.

5.2 Towards Efficient, Safe, and Long-Horizon Agents

Current LLM agents often face computational bottlenecks and struggle with complex, multi-step tasks. Synthesizing insights from the recent survey [14], the field seems to be shifting towards **"LLM-Brained GUI Agents"** that prioritize:

- **Robust Planning and Memory:** A critical research avenue involves enhancing an agent's ability to maintain long-term memory, enabling robust planning for long-horizon tasks that current "stateless" planners frequently fail to complete.
- **On-Device Efficiency:** In contrast to the cloud-dependent and cost-intensive pipelines discussed in Section 4, research is increasingly focusing on model compression for On-Device Deployment. This ensures that test migration can run locally on resource-constrained devices, offering a promising path toward more scalable and reproducible testing.
- **Safety and Privacy:**As agents gain autonomy, ensuring they respect user data confidentiality while interacting with sensitive UI elements becomes paramount.

5.3 Industrial Application: Amazon as a Case Study

The transition from academic prototypes to industrial reality is exemplified by **Amazon's** diverse ecosystem. While academic benchmarks often focus on optimizing performance on limited datasets, industrial implementations prioritize scalability and robustness across heterogeneous environments.

- **Solving Heterogeneity with Vision:** At Amazon's scale, maintaining separate test scripts for every device variation is operationally unsustainable. The adoption of vision-driven approaches (Section 5.1) allows for aligning UI states across different form factors without device-specific code.
- **Semantic Oracles and Adaptation:** Instead of hardcoded assertions, industrial pipelines are moving towards Semantic Test Oracles, where LLMs interpret high-level requirements to verify behavior. For instance, **Amazon Q Developer Agent**[8] demonstrates how autonomous agents can handle code and script migration at the repository level.
- **Business Value:**This industrial adoption suggests that autonomous agents can translate into shorter release cycles and lower operational burdens. By automating the adaptation

of tests to evolving app versions, teams can focus on feature innovation rather than script maintenance.

In summary, LLM-based test migration can be seen as a fundamental paradigm shift from **structure-level mapping** to **intention-level reasoning**. While the field currently faces challenges in visual grounding and semantic fidelity, the convergence of multimodal AI and agent-based planning offers a clear path forward. As demonstrated by industrial pioneers like Amazon, the future of software testing will likely be driven by hybrid agents that are not only "smart" enough to understand intent but also "trustworthy" enough to operate at scale.

6 Conclusion

This report has examined the paradigm shift in automated test migration enabled by Large Language Models, characterizing the transition from rigid syntactic mapping to **semantic intention reasoning**. By synthesizing the **Abstraction–Planning–Grounding** pipeline, we highlighted how modern frameworks successfully decouple test logic from UI implementation details. However, this flexibility introduces critical challenges regarding **visual grounding stability** and **semantic fidelity**, creating a tension between migration adaptability and test correctness. As evidenced by emerging industrial applications at Amazon, the field is evolving from static script reuse toward **autonomous testing agents**. Future success will likely depend on **hybrid neuro-symbolic architectures** that combine the generative power of LLMs with rigorous verification mechanisms to ensure trust and reliability.

References

[1] Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. 2024. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. arXiv:2402.04615 [cs.CV] https://arxiv.org/abs/2402.04615

[2] Benyamin Beyzaei, Saghar Talebipour, Ghazal Rafiei, Nenad Medvidović, and Sam Malek. 2025. Automated Test Transfer across Android Apps using Large Language Models. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA098 (June 2025), 24 pages. doi:10.1145/3728975

[3] Shaoheng Cao, Minxue Pan, Yuanhong Lan, and Xuandong Li. 2025. Intention-Based GUI Test Migration for Mobile Apps using Large Language Models. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA101 (June 2025), 23 pages. doi:10.1145/3728978

[4] Shamit Fatin, Mehbubul Hasan Al-Quvi, Haz Sameen Shahgir, Sukarna Barua, Anindya Iqbal, Sadia Sharmin, Md. Mostofa Akbar, Kallol Kumar Pal, and A. Asif Al Rashid. 2025. LELANTE: LEveraging LLM for Automated ANdroid TEsting. arXiv:2504.20896 [cs.SE] https://arxiv.org/abs/2504.20896

[5] Gang Hu, Linjie Zhu, and Junfeng Yang. 2018. AppFlow: using machine learning to synthesize robust, reusable UI tests. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Lake Buena Vista, FL, USA) *(ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 269–282. doi:10.1145/3236024.3236055

[6] Xiaolei Li, Jialun Cao, Yepang Liu, Shing-Chi Cheung, and Hailong Wang. 2025. ReuseDroid: A VLM-empowered Android UI Test Migrator Boosted by Active Feedback. arXiv:2504.02357 [cs.SE] https://arxiv.org/abs/2504.02357

[7] Jun-Wei Lin, Reyhaneh Jabbarvand, and Sam Malek. 2019. Test Transfer Across Mobile Apps Through Semantic Mapping. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 42–53. doi:10.1109/ASE.2019.00015

[8] Aditya Prakash, Jihye Seo, and Yash Shah. 2025. Streamline code migration using Amazon Nova Premier with an agentic workflow. AWS Machine Learning Blog. https://aws.amazon.com/blogs/machine-learning/streamline-code-migration-using-amazon-nova-premier-with-an-agentic-workflow/ Accessed: 2026-01-11.

[9] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. arXiv:2405.14573 [cs.AI] https://arxiv.org/abs/2405.14573

[10] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing with Large Language Models: Survey, Landscape, and Vision. arXiv:2307.07221 [cs.SE] https://arxiv.org/abs/2307.07221

[11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] https://arxiv.org/abs/2201.11903

[12] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. arXiv:2404.05719 [cs.CV] https://arxiv.org/abs/2404.05719

[13] Shengcheng Yu, Chunrong Fang, Ziyuan Tuo, Quanjun Zhang, Chunyang Chen, Zhenyu Chen, and Zhendong Su. 2025. Vision-Based Mobile App GUI Testing: A Survey. arXiv:2310.13518 [cs.SE] https://arxiv.org/abs/2310.13518

[14] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. Large Language Model-Brained GUI Agents: A Survey. arXiv:2411.18279 [cs.AI] https://arxiv.org/abs/2411.18279

[15] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 70, 20 pages. doi:10.1145/3706598.3713600

[16] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2024. A Survey on Large Language Models for Software Engineering. arXiv:2312.15223 [cs.SE] https://arxiv.org/abs/2312.15223

[17] Yakun Zhang, Chen Liu, Xiaofei Xie, Yun Lin, Jin Song Dong, Dan Hao, and Lu Zhang. 2025. GUI Test Migration via Abstraction and Concretization. *ACM Trans. Softw. Eng. Methodol.* (April 2025). doi:10.1145/3726525 Just Accepted.

[18] Yakun Zhang, Qihao Zhu, Jiwei Yan, Chen Liu, Wenjie Zhang, Yifan Zhao, Dan Hao, and Lu Zhang. 2024. Synthesis-Based Enhancement for GUI Test Case Migration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 869–881. doi:10.1145/3650212.3680327